



Backend API Documentation

Grand Egyptian Museum — Mobile App

Base URL:

<https://gem-backend-production-1ea2.up.railway.app>

General Notes

% **Auth Header**

Authorization: Bearer <token>

% **Language**

Accept-Language: ar OR ?lang=ar

% **Content-Type**

application/json OR multipart/form-data (files)

% **Errors**

```
{ "message": "Error description" }
```

Ø=Ý Authentication /api/auth

POST

/api/auth/register

Register

Create new account

Body:

```
name (String) [required]
email (String) [required]
password (String) [required]
```

Response:

```
201 { "message": "User registered" }
400 User already exists
```

POST

/api/auth/login

Login

Returns JWT token (7 days)

Body:

```
email (String) [required]
password (String) [required]
```

Response:

```
200 { "token": "eyJ..." }
400 Invalid credentials
```

POST

/api/auth/google

Google OAuth Login

Login with Google ID token

Body:

```
tokenId (String) [required]
```

Response:

```
200 { "token": "eyJ..." }
401 Invalid token
```

POST

/api/auth/forgot-password

Forgot Password

Sends 6-digit OTP to email (expires 30 min)

Body:

```
email (String) [required]
```

Response:

```
200 { "message": "Reset code sent" }
404 No user with email
```

POST

/api/auth/verify-reset-code

Verify OTP

Verify the 6-digit code

Body:

```
email (String) [required]
code (String) [required]
```

Response:

```
200 { "message": "Code verified" }
400 Invalid/expired code
```

POST

/api/auth/reset-password

Reset Password

Set new password with OTP

Body:

```
email (String) [required]
code (String) [required]
newPassword (String) [required]
```

Response:

```
200 { "message": "Password updated" }
```

GET

/api/auth/me

Get Current User Ø=Ÿ

Returns user profile (no password)

Response:

200 { _id, name, email, avatar, language, role }

PUT

/api/auth/me

Update Profile Ø=Ÿ

Update name, avatar, language, password

Body:

name (String) [optional]
avatar (String) [optional]
language (String) [optional]
oldPassword (String) [optional]
newPassword (String) [optional]

Response:

200 { "message": "Profile updated", "user": {...} }

POST

/api/auth/me/avatar

Upload Avatar (Cloudinary) Ø=Ÿ

multipart/form-data — max 5MB. Stored permanently on Cloudinary

Body:

avatar (File (image)) [required]

Response:

200 { "message", "avatarUrl": "https://res.cloudinary.com/...", "user" }

PUT

/api/auth/make-admin/:id

Make Admin Ø=Ÿ

Admin only — promote user to admin

Response:

200 Updated user object

Ø&Bum; Artifacts /api/artifacts

GET

/api/artifacts

Get All Artifacts

Returns full list of artifacts

Response:

200 Array of artifacts

GET

/api/artifacts/:id

Get Single Artifact

Get artifact by MongoDB ID

Response:

200 Artifact object

404 Not found

POST

/api/artifacts

Create (Admin) Ø=Ý

Add new artifact

Body:

```
name (String) [required]
description (String) [required]
era (String) [optional]
imageUrl (String) [optional]
model3DUrl (String) [optional]
audioUrl (String) [optional]
videoUrl (String) [optional]
```

Response:

201 Created artifact

PUT

/api/artifacts/:id

Update (Admin) Ø=Ý

Update artifact by ID

Body:

```
name (String) [optional]
description (String) [optional]
imageUrl (String) [optional]
```

Response:

200 Updated artifact

DELETE

/api/artifacts/:id

Delete (Admin) Ø=Ý

Delete artifact by ID

Response:

200 { "message" }

'dp Favorites /api/favorites

POST

/api/favorites/toggle/:itemId

Toggle Favorite +P Ø=Ý

Add or remove in one call. type: Artifact (default) or Event

Body:

type (String) [optional]

Response:

200 { "isFavorited": true/false, "message" }

GET

/api/favorites/my

My Favorites Ø=Ý

Get favorites. Query: ?type=Artifact or ?type=Event

Response:

200 Array of favorites (populated with item data)

GET

/api/favorites/check/:itemId

Check if Favorited Ø=Ý

Query: ?type=Artifact

Response:

200 { "isFavorited": true/false }

GET

/api/favorites/count

Favorites Count Ø=Ý

Query: ?type=Artifact (optional)

Response:

200 { "count": 5 }

POST

/api/favorites/:itemId

Add to Favorites Ø=Ý

Body:

type (String) [optional]

Response:

201 Favorite object

400 Already in favorites

DELETE

/api/favorites/:itemId

Remove from Favorites Ø=Ý

Query: ?type=Artifact

Response:

200 { "message" }

404 Not found

GET

/api/events

Get All Events

Sorted by date ascending

Response:

200 Array of events

POST

/api/events

Create Event (Admin) Ø=Ý

Body:

title (String) [required]
description (String) [required]
date (Date (ISO)) [required]
imageUrl (String) [optional]

Response:

201 Created event

403 Access denied

DELETE

/api/events/:id

Delete Event (Admin) Ø=Ý

Response:

200 { "message" }

Ø<ß« Bookings & Payments /api/bookings

Prices: Egyptian(adult:200,child:100,student:100) | Arab(adult:1450,child:730) | Expatriate(adult:730,child:370) — 14% VAT added

POST

/api/bookings/checkout

Create Checkout Ø=Ý

Creates booking + Paymob payment URL

Body:

```
visitDate (String (ISO)) [required]
nationalityType (String) [required]
tickets (Array) [required]
billingData (Object) [optional]
```

Response:

200 { bookingId, subtotal, tax, total, checkoutUrl }

POST

/api/bookings/verify-payment

Verify Payment Ø=Ý

Call after user returns from Paymob

Body:

```
orderId (String) [required]
transactionId (String) [optional]
```

Response:

200 { "message": "Payment successful", "booking" }

POST

/api/bookings/webhook

Paymob Webhook

Auto-called by Paymob — do NOT call manually

Response:

200 { "message" }

GET

/api/bookings/my-bookings

My Bookings Ø=Ý

User's bookings (newest first)

Response:

200 Array of bookings

GET

/api/bookings/:id

Booking Details Ø=Ý

Response:

200 Booking object

GET

/api/bookings

All Bookings (Admin) Ø=Ý

With user info populated

Response:

200 Array

Ø>Ý AI Features /api/ai

Detection + TTS powered by HuggingFace (egyptian-museum-storyteller). Story-to-Image by Gradio (pharaonic-ai-generator).

POST

/api/ai/ask

AI Chatbot Ø=Ý

Ask about Egyptian history/artifacts

Body:

question (String) [required]

Response:

200 { "answer": "..." }

GET

/api/ai/chats

Chat History Ø=Ý

Response:

200 Array of {question, answer}

POST

/api/ai/detect

Artifact Detection Ø=Ý

multipart/form-data — upload artifact photo. Uses HuggingFace storyteller /detect

Body:

image (File) [required]

Response:

200 { detected, confidence, artifact, rawResult }

503 API not configured / offline

GET

/api/ai/detections

Detection History Ø=Ý

Response:

200 Array of detections

POST

/api/ai/story-to-image

Story !' Image Ø=Ý

Generate pharaonic image from story text (Gradio space). Takes 1-3 min.

Body:

story (String) [required]

Response:

200 { "image": "url", "format": "url", "rawResult" }

500 Generation failed

POST

/api/ai/name-to-cartouche

Name !' Cartouche Ø=Ý

Convert name to hieroglyphic cartouche

Body:

name (String) [required]

Response:

200 { "name", "cartouche": "image_url", "rawResult" }

POST

/api/ai/photo-to-pharaoh

Photo !' Pharaoh Ø=Ý

multipart/form-data — transform selfie to pharaoh

Body:

image (File) [required]

Response:

200 { "pharaohImage": "data:image/...;base64,...", "format": "base64" }

503 API offline

POST

/api/ai/text-to-speech

Full Pipeline (Detect + Story + Audio) Ø=Ý

multipart/form-data — upload artifact photo !' detect + generate story + audio (MP3)

Body:

image (File) [required]

language (String ('ar'|'en')) [optional]

Response:

200 { story, audioBase64, detected, rawResult }

503 API offline

POST

/api/ai/image-to-3d

Image !' 3D (Coming Soon) Ø=Ý

multipart/form-data — placeholder

Body:

image (File) [required]

Response:

202 { "message": "Coming soon", "status": "placeholder" }

Ø=Üä Upload & Ø<ß¬ Videos

POST

/api/upload/video

Upload Video

multipart/form-data — max 100MB

Body:

video (File) [required]
title (String) [optional]

Response:

200 { title, public_id, url, duration }

GET

/api/videos

Get All Videos

Response:

200 Array of videos

POST

/api/videos/add

Add Video Record

Body:

title (String) [required]
public_id (String) [required]
url (String) [required]
duration (Number) [optional]

Response:

200 Video object

Ø<ß Language /api/lang

GET

/api/lang

Get Translations (auto)

Based on Accept-Language header

Response:

200 { lang, translations }

GET

/api/lang/:locale

Get by Locale

e.g. /api/lang/ar

Response:

200 { lang, translations }

GET

/api/lang/all/translations

Get All Languages

Response:

200 { supported, translations: {en:{...}, ar:{...}} }
